

Supervised Bayesian Matrix Factorization

V2 Companion Document: Derivations & Code Mapping

Reference for `Supervised_Bayesian_MF_V2.R`

Andrew Walther

March 2026

Abstract

This companion document provides a complete, self-contained derivation of the Supervised Bayesian Matrix Factorization model and maps every mathematical quantity to its corresponding variable in the V2 R implementation. It is designed to be read alongside `code/Supervised_Bayesian_MF_V2.R` and serves as the authoritative reference for both the theory and its computational realisation. All corrections from the derivation review (R1–R8, documented in `MF_Derivations_UpdateAlgo_REVISED.tex`) are incorporated throughout.

Contents

1	Model Specification & Notation	3
1.1	Index Conventions	3
1.2	Posterior Moment Notation	3
1.3	Genomics Model	3
1.4	Survival Model (Cox Proportional Hazards)	4
1.5	Hierarchical Priors	4
2	Evidence Lower Bound (ELBO)	4
2.1	Variational Family	4
2.2	ELBO Expression	4
3	Cox Partial Likelihood — Taylor Expansion	5
3.1	Expansion Point	5
3.2	Score and Negative Hessian	5
3.3	Working Response	6
4	Update for q_{l_k} — Patient Loadings	6
4.1	Partial Residuals	6
4.2	Genomics Likelihood Contribution	7
4.3	Survival Likelihood Contribution	7
4.4	Combined Coefficients and EBNM Problem	8
5	Update for q_{f_k} — Biological Factors	8
5.1	Derivation	9

6	Update for q_{β_k} — Survival Coefficients	9
6.1	Expanding Over the Posterior of L	10
6.2	Coordinate-Ascent for β_k	10
6.3	Connection to Weighted Least Squares	11
7	Update for τ — Noise Precision	11
7.1	Objective	11
7.2	Expected Squared Residuals	12
7.3	Column-Specific Precision MLE	12
8	Complete Algorithm (Pseudocode $\leftrightarrow$ R Code)	13
8.1	Pseudocode-to-Code Mapping	13
9	Simulation Design	14
9.1	Data Generating Process	14
9.2	Expected Results	15
10	Implementation Notes	15
10.1	Orthogonalisation Approximation [A2]	15
10.2	Taylor-Per-Outer-Iteration Approximation [A6]	16
10.3	Single-Observation EBNM for β	16
10.4	Numerical Floors [A3]	16
A	Errata from Original Derivation Document	16
B	V1-to-V2 Algorithmic Changes	17

1 Model Specification & Notation

1.1 Index Conventions

Symbol	Dimension	Meaning	R Variable
n	scalar	number of samples (patients)	n
p	scalar	number of features (genes / CpG sites)	p
K	scalar	number of latent factors	K
i	$1 \dots n$	sample index	row index
j	$1 \dots p$	feature index	column index
k, k'	$1 \dots K$	factor index	loop variable k
Y	$n \times p$	observed genomics data matrix	Y
L	$n \times K$	loading matrix	EL
F	$p \times K$	factor matrix	EF
β	$K \times 1$	survival coefficient vector	EBeta
τ_j	p -vector	feature-specific noise precision	Tau
(t_i, δ_i)	n -vectors	survival time & event indicator	time, status

1.2 Posterior Moment Notation

Throughout this document, $q(\cdot)$ denotes the variational posterior and $g(\cdot)$ denotes the (learned) prior. The critical distinction is between the *posterior mean* and the *posterior second moment*:

$$\bar{l}_{ik} := \mathbb{E}_q[l_{ik}] \quad (\text{posterior mean}) \quad (1)$$

$$\overline{l_{ik}^2} := \mathbb{E}_q[l_{ik}^2] = \text{Var}_q(l_{ik}) + \bar{l}_{ik}^2 \quad (\text{posterior 2nd moment}) \quad (2)$$

and analogously for $\bar{f}_{jk}$, $\overline{f_{jk}^2}$, $\bar{\beta}_k$, $\overline{\beta_k^2}$. Whenever there is posterior uncertainty, the second moment strictly exceeds the squared mean: $\overline{l_{ik}^2} \geq \bar{l}_{ik}^2$.

Code Mapping (V2.R)

Math	R Variable	Computation in V2.R
$\bar{l}_{ik}$	EL[i,k]	res_L\$posterior\$mean
$\overline{l_{ik}^2}$	EL2[i,k]	res_L\$posterior\$sd^2 + mean^2
$\bar{f}_{jk}$	EF[j,k]	res_F\$posterior\$mean
$\overline{f_{jk}^2}$	EF2[j,k]	res_F\$posterior\$sd^2 + mean^2
$\bar{\beta}_k$	EBeta[k]	res_Beta\$posterior\$mean
$\overline{\beta_k^2}$	EBeta2[k]	res_Beta\$posterior\$sd^2 + mean^2

1.3 Genomics Model

The observed data matrix Y is generated by a low-rank signal plus element-wise Gaussian noise:

$$Y_{n \times p} = L_{n \times K} F_{K \times p}^\top + E_{n \times p}, \quad E_{ij} \sim \mathcal{N}(0, \tau_j^{-1}) \quad (3)$$

Element-wise: $Y_{ij} = \sum_{k=1}^K l_{ik} f_{jk} + \varepsilon_{ij}$. The noise precision τ_j is feature-specific (column-specific), allowing heterogeneous noise levels across genomic features.

1.4 Survival Model (Cox Proportional Hazards)

Patient survival is modelled via a Cox PH model that links the latent loadings $\mathbf{l}_i = (l_{i1}, \dots, l_{iK})^\top$ to the hazard:

$$h(t_i) = h_0(t_i) \exp(\eta_i), \quad \eta_i = \mathbf{l}_i^\top \boldsymbol{\beta} = \sum_{k=1}^K l_{ik} \beta_k \quad (4)$$

where h_0 is an unspecified baseline hazard and $\boldsymbol{\beta}$ quantifies the prognostic contribution of each latent factor.

1.5 Hierarchical Priors

All latent variables receive empirical Bayes priors from the EBNM framework:

$$l_{ik} \stackrel{\text{iid}}{\sim} g_l \in \mathcal{G}_l, \quad f_{jk} \stackrel{\text{iid}}{\sim} g_f \in \mathcal{G}_f, \quad \beta_k \stackrel{\text{iid}}{\sim} g_\beta \in \mathcal{G}_\beta$$

In V2.R, all three use `prior_family = "point_normal"`, i.e., a mixture of a point mass at zero and a normal distribution with data-adaptive variance. This promotes sparsity in all three parameter sets.

2 Evidence Lower Bound (ELBO)

2.1 Variational Family

We adopt a mean-field variational family:

$$q(L, F, \boldsymbol{\beta}) = \prod_{k=1}^K q_{l_k}(\mathbf{l}_{\cdot k}) \prod_{k=1}^K q_{f_k}(\mathbf{f}_{\cdot k}) \prod_{k=1}^K q_{\beta_k}(\beta_k) \quad (5)$$

2.2 ELBO Expression

The ELBO to be *maximised* over posteriors (q_l, q_f, q_β) , priors (g_l, g_f, g_β) , and precision τ is:

$$\begin{aligned} \mathcal{F} = & \underbrace{\mathbb{E}_q[\log P(Y \mid L, F, \tau)]}_{\text{genomics log-lik}} + \underbrace{\mathbb{E}_q[\log P(t, \delta \mid L, \boldsymbol{\beta})]}_{\text{survival log-lik}} \\ & + \sum_k \mathbb{E}_{q_{l_k}} \left[\log \frac{g_{l_k}}{q_{l_k}} \right] + \sum_k \mathbb{E}_{q_{f_k}} \left[\log \frac{g_{f_k}}{q_{f_k}} \right] + \sum_k \mathbb{E}_{q_{\beta_k}} \left[\log \frac{g_{\beta_k}}{q_{\beta_k}} \right] \end{aligned} \quad (6)$$

The key structural property is conditional independence:

$$\log P(Y, t, \delta \mid L, F, \boldsymbol{\beta}, \tau) = \log P(Y \mid L, F, \tau) + \log P(t, \delta \mid L, \boldsymbol{\beta}) \quad (7)$$

This means the genomics and survival likelihoods contribute additively, and each coordinate update can separately aggregate terms from both sources.

Code Mapping (V2.R)

V2.R tracks the *genomics* portion of the ELBO as a monitoring diagnostic (the survival portion is approximated via Taylor expansion and does not have a simple closed-form):

```
# Line 379: ELBO proxy = E_q[log P(Y | L, F, tau)]  
history$elbo_proxy[iter] <- sum(n/2 * log(Tau) - Tau/2 * colSums(R2_bar))
```

This should be non-decreasing across iterations (monotonicity of CAVI for the *genomics* sub-problem).

3 Cox Partial Likelihood — Taylor Expansion

The Cox partial log-likelihood is non-conjugate in both L and β . To handle it within the EBNM framework, we use a second-order Taylor expansion to obtain a locally-Gaussian weighted least squares surrogate.

3.1 Expansion Point

At each outer CAVI iteration, we fix the expansion point at the current posterior means:

$$\hat{\eta}_i = \sum_{k=1}^K \bar{l}_{ik} \bar{\beta}_k \quad (8)$$

Code Mapping (V2.R)

```
# Line 254: eta = EL %*% EBeta (n-vector)  
eta <- as.vector(EL %*% EBeta)
```

3.2 Score and Negative Hessian

The Cox score (gradient) and negative diagonal Hessian at $\hat{\eta}$ are:

$$u_i = \delta_i - \exp(\hat{\eta}_i) \sum_{j: t_j \leq t_i} \frac{\delta_j}{\sum_{m: t_m \geq t_j} \exp(\hat{\eta}_m)} \quad (\text{score}) \quad (9)$$

$$W_{ii} = \exp(\hat{\eta}_i) \sum_{j: t_j \leq t_i} \frac{\delta_j}{\sum_{m: t_m \geq t_j} \exp(\hat{\eta}_m)} \quad (\text{neg-diag Hessian, positive}) \quad (10)$$

Note that $W_{ii} > 0$ for all observed samples, which ensures the quadratic surrogate is concave.

Code Mapping (V2.R)

```
# Lines 70-93: calc_cox_taylor(eta, time, status)  
# Returns: list(u = n-vector score, w = n-vector neg-diagonal Hessian)  
theta <- exp(eta_s) # exp(eta_i)  
risk_sum <- rev(cumsum(rev(theta))) # sum_{m: t_m >= t_i} exp(eta_m)  
h <- status_s / risk_sum # hazard increment
```

```

H <- cumsum(h)           # cumulative hazard
u_s <- status_s - theta * H # score
w_s <- theta * H         # neg-diag Hessian
w_s[w_s < 1e-6] <- 1e-6  # floor to prevent 0-division

```

The sorting by time (`ord <- order(time)`) is required because the at-risk set $\{m : t_m \geq t_j\}$ must be computed via reverse cumulative sums over time-ordered samples.

3.3 Working Response

The 2nd-order Taylor expansion yields:

$$\ell(\boldsymbol{\eta}) \approx \sum_i \left[-\frac{1}{2} W_{ii} (\eta_i - z_i)^2 \right] + C \quad (11)$$

where the **working response** is:

$$z_i = \hat{\eta}_i + \frac{u_i}{W_{ii}} \quad (12)$$

This has the form of a weighted Gaussian log-likelihood: response z_i , precision W_{ii} , mean $\eta_i = \sum_k l_{ik} \beta_k$. The IRLS (Iteratively Reweighted Least Squares) interpretation is: z_i is the “adjusted dependent variable” and W_{ii} is the “working weight.”

Code Mapping (V2.R)

```

# Lines 255-257: working response and weights
taylor <- calc_cox_taylor(eta, time, status)
z      <- eta + taylor$u / taylor$w # n-vector: z_i
w      <- taylor$w                 # n-vector: W_{ii}

```

These are held **fixed** for the entire inner k -loop (standard IRLS-within-VI). If `refresh_taylor = TRUE`, they are recomputed at each k (lines 268–273), giving a “true CAVI” at the cost of extra computation.

4 Update for q_{l_k} — Patient Loadings

4.1 Partial Residuals

To isolate factor k in the coordinate-ascent update, we define:

Genomics partial residual:

$$R_{ij}^{-k} = Y_{ij} - \sum_{k' \neq k} \bar{l}_{ik'} \bar{f}_{jk'} \quad (13)$$

Equivalently, $R^{-k} = Y - \bar{L} \bar{F}^\top + \bar{l}_k \bar{f}_k^\top$ (subtract the full reconstruction, then add back factor k).

Survival partial working response:

$$z_i^{-k} = z_i - \sum_{k' \neq k} \bar{l}_{ik'} \bar{\beta}_{k'} \quad (14)$$

so that $z_i \approx z_i^{-k} + l_{ik} \beta_k$.

Critical observation: z_i^{-k} depends *only* on $k' \neq k$ quantities — it does not depend on l_{ik} or β_k . This means the **same** z_i^{-k} can be reused for both the L -update and the β -update of factor k (see Section 6).

Code Mapping (V2.R)

```
# Lines 290-295: partial residuals inside k-loop
Y_hat <- EL %*% t(EF)                # n x p
R_k   <- Y - Y_hat + outer(EL[,k], EF[,k]) # R^{-k}

eta_no_k <- as.vector(EL %*% EBeta) - EL[,k] * EBeta[k]
z_no_k   <- z - eta_no_k                # z^{-k}
```

[A1] **Gauss-Seidel:** Because **EL** and **EBeta** are updated in-place during the k -loop, the partial residuals for factor k reflect updates from factors $1, \dots, k-1$ within the same iteration.

4.2 Genomics Likelihood Contribution

Expanding the genomics log-likelihood around factor k :

$$\mathbb{E}_{q_f}[\log P(Y \mid L, F, \tau)] \supseteq \sum_{ij} \left[-\frac{1}{2} \tau_j \left(R_{ij}^{-k} - l_{ik} f_{jk} \right)^2 \right] \quad (15)$$

$$= \sum_j -\frac{1}{2} \tau_j \mathbb{E}_q[f_{jk}^2] l_{ik}^2 + \sum_j \tau_j R_{ij}^{-k} \mathbb{E}_q[f_{jk}] l_{ik} + \text{const} \quad (16)$$

Collecting the coefficients of l_{ik}^2 and l_{ik} :

$$A_{ik}^{\text{gen}} = \sum_{j=1}^p \tau_j \overline{f_{jk}^2} \quad (17)$$

$$B_{ik}^{\text{gen}} = \sum_{j=1}^p \tau_j R_{ij}^{-k} \bar{f}_{jk} \quad (18)$$

4.3 Survival Likelihood Contribution

From Eq. (11), isolating the terms in l_{ik} :

$$-\frac{1}{2} W_{ii} \left(z_i^{-k} - l_{ik} \beta_k \right)^2 = -\frac{1}{2} W_{ii} \mathbb{E}_q[\beta_k^2] l_{ik}^2 + W_{ii} z_i^{-k} \mathbb{E}_q[\beta_k] l_{ik} + \text{const} \quad (19)$$

The coefficients are:

$$A_{ik}^{\text{surv}} = W_{ii} \overline{\beta_k^2} \quad (20)$$

$$B_{ik}^{\text{surv}} = W_{ii} z_i^{-k} \bar{\beta}_k \quad (21)$$

4.4 Combined Coefficients and EBNM Problem

The total quadratic form in l_{ik} is $-\frac{1}{2}A_{ik}l_{ik}^2 + B_{ik}l_{ik}$, where:

$$A_{ik} = \sum_{j=1}^p \tau_j \overline{f_{jk}^2} + W_{ii} \overline{\beta_k^2} \quad (22)$$

$$B_{ik} = \sum_{j=1}^p \tau_j R_{ij}^{-k} \bar{f}_{jk} + W_{ii} z_i^{-k} \bar{\beta}_k \quad (23)$$

Dimensions: A_{ik} is an n -vector (varies across samples i via W_{ii}). The first term $\sum_j \tau_j \overline{f_{jk}^2}$ is a scalar (shared across all i); the second term $W_{ii} \overline{\beta_k^2}$ is sample-specific.

EBNM Problem

For each sample $i = 1, \dots, n$, solve:

$$x_i = \frac{B_{ik}}{A_{ik}}, \quad s_i = \frac{1}{\sqrt{A_{ik}}} \implies (\hat{g}_{l_k}, \hat{q}_{l_k}) = \text{EBNM}(\mathbf{x}, \mathbf{s})$$

The EBNM solver returns: $\bar{l}_{ik} = \mathbb{E}_{\hat{q}}[l_{ik}]$ and $\bar{l}_{ik}^2 = \text{Var}_{\hat{q}}(l_{ik}) + \bar{l}_{ik}^2$.

Code Mapping (V2.R)

```
# Lines 310-321: L update for factor k
A_L <- sum(Tau * EF2[, k]) + w * EBeta2[k] # n-vector
A_L <- pmax(A_L, 1e-10) # [A3] floor

B_L_gen <- as.vector(R_k %*% (Tau * EF[, k])) # n-vector
B_L_surv <- w * z_no_k * EBeta[k] # n-vector
B_L <- B_L_gen + B_L_surv

res_L <- ebnm(x = B_L/A_L, s = 1/sqrt(A_L),
              prior_family = "point_normal")
EL[, k] <- res_L$posterior$mean # l_bar
EL2[, k] <- res_L$posterior$sd^2 + res_L$posterior$mean^2 # E[l^2]
```

Efficiency note: `R_k %*% (Tau * EF[,k])` computes $\sum_j \tau_j R_{ij}^{-k} \bar{f}_{jk}$ as a single matrix-vector product ($n \times p$ times $p \times 1$), avoiding the creation of an intermediate $n \times p$ matrix from `sweep()`.

5 Update for q_{f_k} — Biological Factors

F appears **only** in the genomics term of the likelihood (not in the survival model), so the F -update is identical to the standard EBMF update.

5.1 Derivation

Using the same partial residual R_{ij}^{-k} from Eq. (13) and taking the expectation over q_l (using updated values from the L -step):

$$A_{jk} = \sum_{i=1}^n \tau_j \bar{l}_{ik}^2 = \tau_j \sum_{i=1}^n \bar{l}_{ik}^2 \quad (24)$$

$$B_{jk} = \sum_{i=1}^n \tau_j R_{ij}^{-k} \bar{l}_{ik} = \tau_j \sum_{i=1}^n R_{ij}^{-k} \bar{l}_{ik} \quad (25)$$

Dimensions: A_{jk} is a p -vector. Since we assume column-specific precision (τ_j shared across all i), the scalar $\sum_i \bar{l}_{ik}^2$ factors out and $A_{jk} = \tau_j \cdot (\text{scalar})$.

EBNM Problem

For each feature $j = 1, \dots, p$, solve:

$$x_j = \frac{B_{jk}}{A_{jk}} = \frac{\sum_i R_{ij}^{-k} \bar{l}_{ik}}{\sum_i \bar{l}_{ik}^2}, \quad s_j = \frac{1}{\sqrt{A_{jk}}} \implies (\hat{g}_{f_k}, \hat{q}_{f_k}) = \text{EBNM}(\mathbf{x}, \mathbf{s})$$

Note that x_j does not depend on τ_j (it cancels in B/A), but $s_j = 1/\sqrt{\tau_j \sum_i \bar{l}_{ik}^2}$ does: higher-precision features get tighter pseudo-standard-deviations, contributing more signal.

Code Mapping (V2.R)

```
# Lines 334-340: F update for factor k
sum_EL2_k <- sum(EL2[, k]) # scalar
A_F <- pmax(Tau * sum_EL2_k, 1e-10) # p-vector [A3]
B_F <- Tau * as.vector(t(R_k) %*% EL[, k]) # p-vector

res_F <- ebnm(x = B_F/A_F, s = 1/sqrt(A_F),
              prior_family = "point_normal")
EF[, k] <- res_F$posterior$mean
EF2[, k] <- res_F$posterior$sd^2 + res_F$posterior$mean^2
```

Note: `t(R_k) %*% EL[,k]` computes $\sum_i R_{ij}^{-k} \bar{l}_{ik}$ as a $p \times n$ times $n \times 1$ product.

6 Update for q_{β_k} — Survival Coefficients

β appears only in the survival term, so the relevant ELBO component is:

$$\mathcal{F}(q_{\beta}, g_{\beta}) = \mathbb{E}_{q_l, q_{\beta}} [\log P(t, \delta \mid L, \beta)] + \sum_k \mathbb{E}_{q_{\beta_k}} \left[\log \frac{g_{\beta_k}}{q_{\beta_k}} \right] \quad (26)$$

6.1 Expanding Over the Posterior of L

Starting from the Taylor surrogate (Eq. 11):

$$\mathbb{E}_{q_l, q_\beta} \left[\sum_i -\frac{1}{2} W_{ii} (z_i - \eta_i)^2 \right], \quad \eta_i = \sum_k l_{ik} \beta_k \quad (27)$$

The key expansion is:

$$\mathbb{E}_{q_l} \left[\left(\sum_k l_{ik} \beta_k \right)^2 \right] = \left(\sum_k \bar{l}_{ik} \beta_k \right)^2 + \sum_k \text{Var}_q(l_{ik}) \beta_k^2 \quad (28)$$

This follows from mean-field independence ($l_{ik} \perp l_{ik'}$ under q) and the identity $\mathbb{E}[l^2] = \text{Var} + (\mathbb{E}[l])^2$. Substituting into the objective and rearranging:

$$\begin{aligned} \mathcal{F}(q_\beta, g_\beta) &= \mathbb{E}_{q_\beta} \left[\sum_i -\frac{1}{2} W_{ii} \left(z_i - \sum_k \bar{l}_{ik} \beta_k \right)^2 \right] \\ &\quad - \mathbb{E}_{q_\beta} \left[\sum_k \frac{1}{2} \left(\sum_i W_{ii} \text{Var}_q(l_{ik}) \right) \beta_k^2 \right] + \text{KL terms} \end{aligned} \quad (29)$$

6.2 Coordinate-Ascent for β_k

Fixing all $\beta_{k'}, k' \neq k$ and collecting terms in β_k :

- From term 1: $-\frac{1}{2} [\sum_i W_{ii} \bar{l}_{ik}^2] \beta_k^2 + [\sum_i W_{ii} z_i^{-k} \bar{l}_{ik}] \beta_k$
- From term 2: $-\frac{1}{2} [\sum_i W_{ii} \text{Var}_q(l_{ik})] \beta_k^2$

Combining and using $\overline{l_{ik}^2} = \text{Var}_q(l_{ik}) + \bar{l}_{ik}^2$:

$$\boxed{A_k = \sum_{i=1}^n W_{ii} \overline{l_{ik}^2}} \quad (30)$$

$$\boxed{B_k = \sum_{i=1}^n W_{ii} z_i^{-k} \bar{l}_{ik}} \quad (31)$$

Dimensions: A_k and B_k are both *scalars* (summed over all n samples for a single factor k).

Error-in-variables correction. Using $\overline{l_{ik}^2}$ (which includes $\text{Var}_q(l_{ik})$) in A_k rather than $\bar{l}_{ik}^2$ naturally inflates the effective noise variance, preventing β_k from overfitting to uncertain loadings. This is the Bayesian analogue of errors-in-variables regression.

Reuse of z_i^{-k} . The partial working response $z_i^{-k} = z_i - \sum_{k' \neq k} \bar{l}_{ik'} \bar{\beta}_{k'}$ is the **same** quantity computed in Section 4.1 for the L -update. It can be reused because it depends only on $k' \neq k$ quantities.

EBNM Problem

For factor k , solve the EBNM problem with a *single* pseudo-observation:

$$x_k = \frac{B_k}{A_k}, \quad s_k = \frac{1}{\sqrt{A_k}} \quad \implies \quad (\hat{g}_{\beta_k}, \hat{q}_{\beta_k}) = \text{EBNM}(x_k, s_k)$$

The EBNM solver estimates the prior g_{β_k} from this single observation. While a single data point limits the information available for prior estimation, the point-normal family has only two parameters (mixing proportion π and variance σ^2), and the shrinkage is mainly driven by s_k . See Section 10 for further discussion.

Code Mapping (V2.R)

```
# Lines 356-361: Beta update for factor k
A_Beta <- max(sum(w * EL2[, k]), 1e-10)      # scalar [A3]
B_Beta <- sum(w * z_no_k * EL[, k])          # scalar

res_Beta <- ebnm(x = B_Beta/A_Beta, s = 1/sqrt(A_Beta),
                prior_family = "point_normal")
EBeta[k] <- res_Beta$posterior$mean
EBeta2[k] <- res_Beta$posterior$sd^2 + res_Beta$posterior$mean^2
```

z_no_k reuse: The variable `z_no_k` computed at line 295 is reused here at line 357. No recomputation is needed.

6.3 Connection to Weighted Least Squares

The first term of Eq. (29) has the form of a weighted regression:

$$z_i = \sum_k \bar{l}_{ik} \beta_k + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, W_{ii}^{-1})$$

The WLS estimator (ignoring priors and variance correction) would be $\hat{\beta} = (\bar{L}^\top W \bar{L})^{-1} \bar{L}^\top W \mathbf{z}$. The coordinate-ascent EBNM update generalises this by: (i) accounting for uncertainty in L via $\bar{l}_{ik}^2$, and (ii) applying EBNM shrinkage priors.

7 Update for τ — Noise Precision

7.1 Objective

The ELBO components involving τ are:

$$\mathcal{F}(\tau) = \frac{1}{2} \sum_{ij} [\log \tau_j - \tau_j \bar{R}_{ij}^2] + C \quad (32)$$

Note the **minus sign** before $\tau_j \bar{R}_{ij}^2$ (see REVISED.tex correction [R7]).

7.2 Expected Squared Residuals

Under the mean-field assumption, the expected squared residual is:

$$\bar{R}_{ij}^2 := \mathbb{E}_{q_i, q_f} \left[\left(Y_{ij} - \sum_k l_{ik} f_{jk} \right)^2 \right] = \underbrace{\left(Y_{ij} - \sum_k \bar{l}_{ik} \bar{f}_{jk} \right)^2}_{\text{squared residual at posterior means}} + \underbrace{\sum_k \left[\overline{l_{ik}^2 f_{jk}^2} - \bar{l}_{ik}^2 \bar{f}_{jk}^2 \right]}_{\text{variance inflation}} \quad (33)$$

Derivation of the variance term:

$$\mathbb{E} \left[\left(\sum_k l_{ik} f_{jk} \right)^2 \right] = \sum_k \mathbb{E}[l_{ik}^2] \mathbb{E}[f_{jk}^2] + \sum_{k \neq k'} \mathbb{E}[l_{ik}] \mathbb{E}[f_{jk}] \mathbb{E}[l_{ik'}] \mathbb{E}[f_{jk'}] \quad (34)$$

$$= \sum_k \overline{l_{ik}^2 f_{jk}^2} + \sum_{k \neq k'} \bar{l}_{ik} \bar{f}_{jk} \bar{l}_{ik'} \bar{f}_{jk'} \quad (35)$$

Meanwhile:

$$\left(\sum_k \bar{l}_{ik} \bar{f}_{jk} \right)^2 = \sum_k \bar{l}_{ik}^2 \bar{f}_{jk}^2 + \sum_{k \neq k'} \bar{l}_{ik} \bar{f}_{jk} \bar{l}_{ik'} \bar{f}_{jk'}$$

Subtracting, the cross-terms cancel, leaving $\sum_k [\overline{l_{ik}^2 f_{jk}^2} - \bar{l}_{ik}^2 \bar{f}_{jk}^2]$, which equals $\sum_k \text{Var}_q(l_{ik}) \mathbb{E}_q[f_{jk}^2] + \bar{l}_{ik}^2 \text{Var}_q(f_{jk}) + \text{Var}_q(l_{ik}) \text{Var}_q(f_{jk})$.

7.3 Column-Specific Precision MLE

Setting $\partial \mathcal{F} / \partial \tau_j = 0$:

$$\hat{\tau}_j = \frac{n}{\sum_{i=1}^n \bar{R}_{ij}^2} \quad (36)$$

Code Mapping (V2.R)

```
# Lines 374-376: Tau update
Var_Term <- (EL2 %*% t(EF2)) - (EL^2 %*% t(EF^2)) # n x p
R2_bar   <- (Y - EL %*% t(EF))^2 + Var_Term         # n x p
Tau      <- n / pmax(colSums(R2_bar), n * 1e-8)    # p-vector
```

Matrix operations:

- `EL2 %*% t(EF2)` computes $\sum_k \overline{l_{ik}^2 f_{jk}^2}$ ($n \times p$).
- `EL^2 %*% t(EF^2)` computes $\sum_k \bar{l}_{ik}^2 \bar{f}_{jk}^2$ ($n \times p$).
- Their difference is the variance inflation $\sum_k [\dots]$ in Eq. (33).
- `colSums(R2_bar)` computes $\sum_i \bar{R}_{ij}^2$ (a p -vector).

8 Complete Algorithm (Pseudocode $\leftrightarrow$ R Code)

Supervised MF — CAVI Algorithm

Inputs: $Y_{n \times p}$, $(t_i, \delta_i)_{i=1}^n$, K , `max_iter`, `tol`

Initialize:

1. SVD of Y : $\bar{L} = U\sqrt{D}$, $\bar{F} = V\sqrt{D}$; $\bar{l}^2 \leftarrow \bar{l}^2$, $\bar{f}^2 \leftarrow \bar{f}^2$.
2. Warm-start $\bar{\beta}$: fit `coxph()` on initial loadings.
3. $\tau_j \leftarrow 1/\text{Var}(Y_{\cdot j})$ for each column j .

For $b = 1, 2, \dots, \text{max_iter}$:

1. **Cox Taylor expansion:** $\hat{\eta} = \bar{L}\bar{\beta}$; $(u_i, W_{ii}) \leftarrow \text{CoxTaylor}(\hat{\eta}, t, \delta)$; $z_i = \hat{\eta}_i + u_i/W_{ii}$.
2. **For** $k = 1, \dots, K$:
 - (a) **Compute partial residuals:** R^{-k} , z_i^{-k} .
 - (b) **Update** q_{l_k} : $A_{ik}, B_{ik} \rightarrow \text{EBNM} \rightarrow \bar{l}_{ik}, \bar{l}_{ik}^2$.
 - (c) **Update** q_{f_k} : $A_{jk}, B_{jk} \rightarrow \text{EBNM} \rightarrow \bar{f}_{jk}, \bar{f}_{jk}^2$.
 - (d) **Update** q_{β_k} : $A_k, B_k \rightarrow \text{EBNM} \rightarrow \bar{\beta}_k, \bar{\beta}_k^2$.
3. **Update** $\hat{\tau}_j$: $\bar{R}_{ij}^2 \rightarrow \hat{\tau}_j = n / \sum_i \bar{R}_{ij}^2$.
4. **Convergence:** If $\text{mean}|\Delta L| < \text{tol}$ **and** $\text{mean}|\Delta \beta| < \text{tol}$, stop.

Return: $(\bar{L}, \bar{F}, \bar{\beta}, \bar{\beta}^2, \hat{\tau}, \text{history})$.

8.1 Pseudocode-to-Code Mapping

Algorithm Step	V2.R Code	Line(s)
SVD initialisation	<code>svd(Y, nu=K, nv=K)</code>	187–195
Cox warm-start for β	<code>coxph(...)</code>	198–212
τ init from column variance	<code>1.0 / col_var</code>	217–218
$\hat{\eta} = \bar{L}\bar{\beta}$	<code>EL %% EBeta</code>	254
$(u, w) = \text{CoxTaylor}$	<code>calc_cox_taylor(eta, ...)</code>	255
$z_i = \hat{\eta}_i + u_i/W_{ii}$	<code>eta + taylor\$u / taylor\$w</code>	256
R_{ij}^{-k}	<code>Y - Y_hat + outer(EL[,k], EF[,k])</code>	290–291
z_i^{-k}	<code>z - eta_no_k</code>	294–295
A_{ik} (loadings)	<code>sum(Tau*EF2[,k]) + w*EBeta2[k]</code>	310

Algorithm Step	V2.R Code	Line(s)
B_{ik}^{gen}	<code>R_k %*% (Tau * EF[,k])</code>	315
B_{ik}^{surv}	<code>w * z_no_k * EBeta[k]</code>	316
EBNM for l_k	<code>ebnm(x=B_L/A_L, s=...)</code>	319
A_{jk} (factors)	<code>Tau * sum_EL2_k</code>	335
B_{jk}	<code>Tau * t(R_k) %*% EL[,k]</code>	336
EBNM for f_k	<code>ebnm(x=B_F/A_F, s=...)</code>	338
A_k (survival coeff)	<code>sum(w * EL2[,k])</code>	356
B_k	<code>sum(w * z_no_k * EL[,k])</code>	357
EBNM for β_k	<code>ebnm(x=B_Beta/A_Beta, ...)</code>	359
Variance term	<code>EL2%*%t(EF2) - EL^2%*%t(EF^2)</code>	374
$\bar{R}_{ij}^2$	<code>(Y-EL%*%t(EF))^2 + Var_Term</code>	375
$\hat{\tau}_j$	<code>n / colSums(R2_bar)</code>	376
ELBO proxy	<code>sum(n/2*log(Tau) - ...)</code>	379
ΔL convergence	<code>mean(abs(EL - EL_old))</code>	400
$\Delta \beta$ convergence	<code>mean(abs(EBeta - EBeta_old))</code>	401
Dual check	<code>delta_L < tol && delta_Beta < tol</code>	410

9 Simulation Design

The V2.R script includes a synthetic benchmark to validate the algorithm.

9.1 Data Generating Process

1. **Loadings:** $L_{n \times K} \sim \mathcal{N}(0, 1)$ i.i.d., with $n = 250$, $K = 5$.
2. **Factors:** $F_{p \times K}$ is sparse: for each factor k , 5% of features are active ($\sim \mathcal{N}(0, 25)$); the remaining 95% are exactly zero. This yields $p = 1000$ features.
3. **Genomics data:** $Y = LF^\top + E$, $E_{ij} \sim \mathcal{N}(0, 1)$.
4. **Survival times:** Weibull baseline with scale $\lambda = 0.01$, shape $\alpha = 1.5$, and Cox link:

$$T_i = \left(\frac{-\log(U_i)}{\lambda \exp(\eta_i^{\text{true}})} \right)^{1/\alpha}, \quad U_i \sim \text{Uniform}(0, 1)$$

where $\eta_i^{\text{true}} = L_i^\top \beta^{\text{true}}$.

5. **True survival coefficients:** $\beta^{\text{true}} = (1.5, -1.2, 0.8, -0.5, 0.0)$. Factor 5 ($\beta_5 = 0$) is purely structural (no prognostic value).
6. **Censoring:** Independent exponential censoring with rate 1/50: $C_i \sim \text{Exp}(1/50)$. Observed time $t_i = \min(T_i, C_i)$; event $\delta_i = \mathbb{I}(T_i \leq C_i)$.

Code Mapping (V2.R)

```
# Lines 519-544: sim_data_fn()
L      <- matrix(rnorm(n * k), n, k)           # Step 1
F_mat <- matrix(0, p, k)                       # Step 2
for (i in 1:k) {
  active <- sample(1:p, round(p * 0.05))
  F_mat[active, i] <- rnorm(length(active), 0, 5)
}
Y <- L %*% t(F_mat) + matrix(rnorm(n * p), n, p) # Step 3
Beta      <- c(1.5, -1.2, 0.8, -0.5, 0.0)      # Step 5
eta_true <- as.vector(L %*% Beta)
raw_times <- (-log(runif(n)) / (0.01 * exp(eta_true)))^(1/1.5) # Step 4
cens_times <- rexp(n, rate = 1/50)             # Step 6
```

9.2 Expected Results

- **RMSE** should converge near 1.0 (the noise standard deviation).
- **ELBO proxy** should be non-decreasing across iterations.
- **Estimated** β should recover the sign pattern $(+, -, +, -, 0)$ with magnitudes proportional to truth.
- **Supervised C-index** should exceed Top-5 PCA C-index.
- **Factor 5** ($\beta_5^{\text{true}} = 0$) should have $\hat{\beta}_5 \approx 0$ (structural factor correctly identified as non-prognostic).

10 Implementation Notes

10.1 Orthogonalisation Approximation [A2]

When `orthogonalize = TRUE` (default **OFF**), every 10 iterations the code applies an SVD rotation to $\bar{L}$ for factor identifiability:

$$\begin{aligned}\bar{L} &\leftarrow U_L D_L, & \bar{F} &\leftarrow \bar{F} V_L \\ \bar{l}^2 &\leftarrow \bar{l}^2, & \bar{f}^2 &\leftarrow \bar{f}^2\end{aligned}$$

This resets the second moments to squared means, **discarding posterior variance**. The true second moment $\bar{l}_{ik}^2 = \text{Var}_q + \bar{l}_{ik}^2 \geq \bar{l}_{ik}^2$ is only recovered at the next EBNM call. Consequences:

- Transient underestimate of posterior uncertainty every 10 iterations.
- The β precision A_k is temporarily biased low (less shrinkage).
- The τ update omits variance inflation (biased high).

These effects are mild and self-correcting. The orthogonalisation is off by default because Point-Normal EBNM already promotes factor distinctness through sparsity.

10.2 Taylor-Per-Outer-Iteration Approximation [A6]

By default (`refresh_taylor = FALSE`), (z, w) are computed once per outer iteration using $\hat{\eta}$ from the *start* of that iteration. As $\bar{L}$ and $\bar{\beta}$ change within the inner k -loop, the true linear predictor shifts but z and w remain stale.

This is the standard IRLS-within-VI approximation: the outer loop restores exactness by re-computing (z, w) at the next iteration. Setting `refresh_taylor = TRUE` recomputes them at every k , giving a more accurate but slower inner loop.

10.3 Single-Observation EBNM for β

Each β_k update passes a *single* pseudo-observation (x_k, s_k) to EBNM. With the point-normal prior family (two parameters: π_0 and σ^2), the solver has limited data for prior estimation. In practice:

- If $|x_k|/s_k \gg 1$ (strong signal), the posterior mean is close to x_k .
- If $|x_k|/s_k \ll 1$ (weak signal), EBNM shrinks toward zero ($\pi_0 \rightarrow 1$), correctly identifying non-prognostic factors.
- The prior is re-estimated at every CAVI iteration, allowing adaptation as the loadings L stabilise.

10.4 Numerical Floors [A3]

All EBNM precision inputs receive a floor of 10^{-10} via `pmax(..., 1e-10)` to prevent:

- Division by zero in $s = 1/\sqrt{A}$.
- Infinite pseudo-observations $x = B/A$ when $A \approx 0$.
- Degenerate EBNM solves with unbounded prior variance.

Similarly, the τ denominator is floored at $n \times 10^{-8}$ to prevent $\tau_j \rightarrow \infty$ for near-zero-variance features.

A Errata from Original Derivation Document

The following errors were found during a detailed review of the original derivation document (MF_Derivations_UpdateAlgo_2_12_26.pdf) and are corrected in MF_Derivations_UpdateAlgo_REVISED.tex.

ID	Location	Error in Original	Correction
R1	Sec. 1A	Wrong dimension subscripts on Y, L, F, E	$Y_{n \times p} = L_{n \times K} F_{K \times p}^\top + E_{n \times p}$
R2	Eq. 22	z_i^{-k} defined with $f_{jk'}$ instead of $\beta_{k'}$	$z_i^{-k} = z_i - \sum_{k' \neq k} \bar{l}_{ik'} \bar{\beta}_{k'}$
R3	Eqs. 32–35	f_{jk}^2, β_k^2 (raw squares) where second moments $\mathbb{E}_q[\cdot]$ are needed	Replaced with $\overline{f_{jk}^2}, \overline{\beta_k^2}$
R4	Eqs. 47–48	EBNM inputs subscripted i (sample)	Corrected to j (feature)
R5	Eq. 62	β_k missing square in expansion	$\sum_k l_{ik}^2 \beta_k^2$
R6	Sec. 6	β update as generic WLS; no EBNM form	Derived A_k, B_k for EBNM
R7	Eq. 83	Sign error: $\log \tau + \tau \bar{R}^2$	Corrected to $\log \tau - \tau \bar{R}^2$
R8	Eq. 85	$\bar{l}_{jk}^2$ (wrong index j)	Corrected to $\bar{l}_{ik}^2$

B V1-to-V2 Algorithmic Changes

ID	Issue in V1	V1 Behaviour	V2 Fix
A1	Stale η in z^{-k}	<code>z.no_k</code> computed from start-of-iter η only	Recompute <code>eta.no_k</code> from current <code>EL</code> , <code>EBeta</code> inside k -loop
A2	Orthogonalisation	Always ON; resets <code>EL2/EF2</code>	Behind <code>orthogonalize=FALSE</code> flag (default OFF)
A3	No precision floor	<code>A.L</code> , <code>A.F</code> , <code>A.Beta</code> could be ≤ 0	<code>pmax(..., 1e-10)</code> on all three
A4	Convergence on L only	Only <code>delta.L</code> checked	Both <code>delta.L</code> and <code>delta.Beta</code> must be $< \text{tol}$
A5	No ELBO tracking	Only RMSE recorded	<code>elbo.proxy</code> = genomics log-likelihood tracked
A6	Taylor fixed per iter	(z, w) fixed for all k	<code>refresh.taylor</code> flag for optional per- k refresh